

SE446: Big Data Engineering

Week 8A: Spark on the Cluster — Deployment & Performance Tuning

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

Learning Objectives

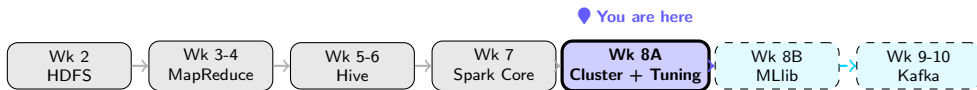
By the end of this session you will be able to:

- 1 Submit Spark jobs to a YARN cluster using `spark-submit`
- 2 Distinguish `client` vs `cluster` deploy modes
- 3 Read the Spark Web UI: Jobs, Stages, Tasks, Storage, Executors
- 4 Apply caching with `.cache()` and `.persist()`
- 5 Tune partition count for optimal parallelism
- 6 Use broadcast joins for small lookup tables

Dataset

Chicago Crimes (HDFS) — same dataset from Week 7

The Big Data Journey — Where Are We?



What you already know

- RDDs, DataFrames, Spark SQL
- PySpark in **interactive shell**
- Ran jobs locally or on the shell

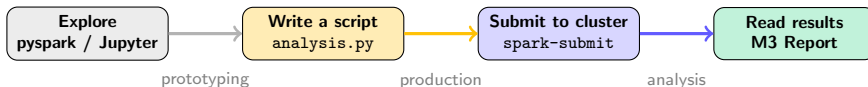


What changes today

- Submit **real jobs** to the cluster
- Read the **Spark Web UI**
- Make jobs **fast** (tuning)
- **M3 milestone** uses these skills

Outline

From Exploration to Submission — Your M3 Workflow



>_ PySpark Shell / Jupyter

- Type code, see results instantly
- Great for **exploring** the dataset
- Driver runs on **your machine**
- Cannot run unattended



spark-submit on YARN

- Runs a `.py` script as a batch job
- Works on the **cluster's resources**
- Reproducible — versioned in GitHub
- Required for **M3 submission**

The Rule

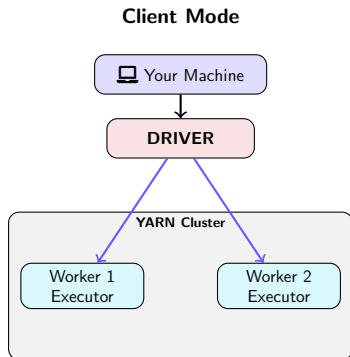
Prototype in Jupyter → polish to `.py` → `spark-submit` → submit results for M3.

spark-submit Syntax

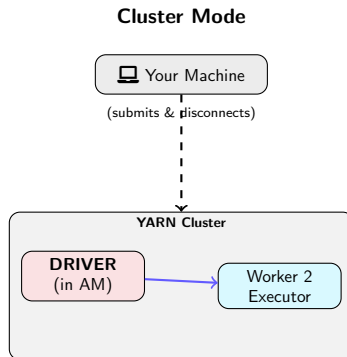
```
spark-submit \  
  --master yarn \  
  --deploy-mode client \  
  --num-executors 2 \  
  --executor-memory 1g \  
  --executor-cores 2 \  
  --driver-memory 512m \  
  --conf spark.sql.shuffle.partitions=4 \  
  crime_analysis.py
```

Flag	Purpose
--master yarn	Run on YARN cluster
--deploy-mode	Where Driver runs (client/cluster)
--num-executors	Number of executor JVMs
--executor-memory	RAM per executor
--executor-cores	CPU cores per executor
--conf	Custom Spark configuration

Deploy Mode: client vs cluster



- ✓ See output live
- ✓ Great for debugging



- ✓ Survives SSH disconnect
- ✓ Best for production

Understanding Deploy Modes — Key Concepts



Client Mode — Driver on Your Machine

- The **Driver** process runs on the machine where you typed `spark-submit`
- Your terminal stays **attached** — you see `print()` output live
- If your SSH session drops, the **entire job dies** (Driver is gone)
- The Driver communicates with YARN to request Executors on Worker nodes
- Best for: **development, debugging, interactive exploration**



Cluster Mode — Driver Inside YARN

- YARN launches the Driver inside an **Application Master (AM)** container on a Worker
- Your machine **submits and disconnects** — the job runs independently
- Job **survives** SSH disconnects, laptop shutdowns, network issues
- Output goes to YARN logs (retrieve via `yarn logs -applicationId <id>`)
- Best for: **production jobs, long-running pipelines, scheduled tasks**

Setting Up Spark Client on Your Computer

1. Install Spark & Set Environment

Download the **same Spark version** as your cluster from `spark.apache.org`, then:

```
export SPARK_HOME=/opt/spark           # where you extracted Spark
export PATH=$SPARK_HOME/bin:$PATH
export HADOOP_CONF_DIR=/etc/hadoop/conf
```

2. Point to Your Cluster

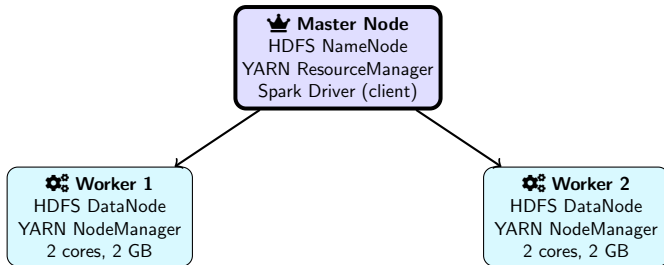
Copy these three files from the cluster into `$HADOOP_CONF_DIR`: `core-site.xml`, `hdfs-site.xml`, `yarn-site.xml`. Your machine must be able to reach the cluster nodes over the network.

3. Submit in Client Mode

```
spark-submit --master yarn --deploy-mode client \
  --num-executors 2 --executor-memory 1g my_analysis.py
```

Your laptop is the **Driver** → `print()` output appears in your terminal.

Our Cluster Configuration



Spark Web UI: <http://master:4040> — YARN UI: <http://master:8088>

Cluster Resources

Total: 4 cores, 4 GB RAM
Request $\leq$ 1g per executor

Dataset

`hdfs:///data/chicago_crimes.csv`
~10,000 rows

Understanding the Cluster — Roles Explained



Master Node — The Coordinator

- **HDFS NameNode**: knows where every data block is stored across the cluster
- **YARN ResourceManager**: decides which Worker gets how many containers (CPU + RAM)
- **Spark Driver** (in client mode): your program's "brain" that plans the execution
- Does **not** store data or run tasks itself — it only coordinates



Worker Nodes — The Muscle

- **HDFS DataNode**: stores actual data blocks (crimes.csv chunks)
- **YARN NodeManager**: manages containers, reports resources to ResourceManager
- **Spark Executors**: JVM processes that run your tasks (map, filter, groupBy, etc.)
- Each worker has **2 cores, 2 GB** — so request $\leq 1g$ per executor

Example Script: Crime Counts

```
# crime_counts.py
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count

spark = SparkSession.builder \
    .appName("CrimeCounts") \
    .getOrCreate()

df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)

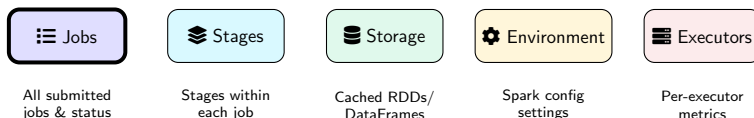
result = df.groupBy("Primary Type") \
    .agg(count("*").alias("cnt")) \
    .orderBy(col("cnt").desc())

result.show(10)
spark.stop()
```

Submit to YARN

```
spark-submit --master yarn --deploy-mode client crime_counts.py
```


Spark Web UI — Overview



While App Runs

`http://master:4040`

After App Finishes

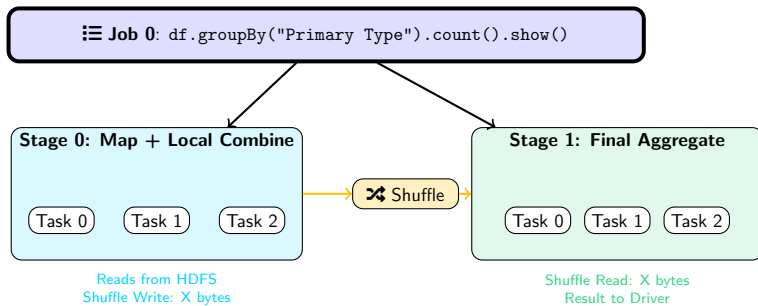
YARN proxy:

`http://master:8088/proxy/<app-id>/`

Key Principle

Each **action** creates a **job**. Each job has one or more **stages** (separated by shuffles). Each stage has multiple **tasks** (one per partition).

Jobs → Stages → Tasks



Minimize Shuffle!

Shuffle = data transfer across the network. **Shuffle bytes** in the Stages tab is the #1 metric to watch for performance.

Understanding Jobs, Stages, and Tasks

☰ Jobs — Triggered by Actions

- Every **action** (`.show()`, `.count()`, `.collect()`, `.write`) creates a new **Job**
- Transformations (`.filter()`, `.groupBy()`) are lazy — they do **not** trigger Jobs
- In the Web UI, each Job is numbered (Job 0, Job 1, ...)

📦 Stages — Separated by Shuffles

- A Job is split into **Stages** at every **shuffle boundary** (e.g., `groupBy`, `join`)
- **Stage 0** (pre-shuffle): reads data, applies filters, does local aggregation
- **Stage 1** (post-shuffle): receives shuffled data, computes final result
- Fewer stages = fewer shuffles = **better performance**

📄 Tasks — The Smallest Unit of Work

- Each stage launches **one task per partition** (e.g., 4 partitions = 4 tasks)

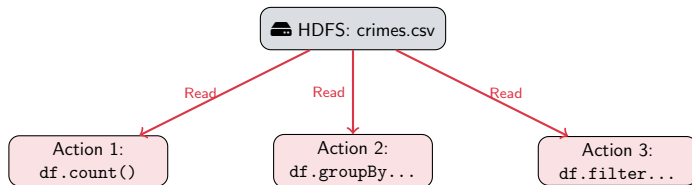
Spotting Performance Problems in the Web UI

Symptom	Where to Look	Cause & Fix
One task 10× slower	Tasks tab — Duration column	Data skew: one partition has much more data. Fix: <code>repartition()</code> or salting.
High GC Time	Executors tab	Memory pressure: increase <code>executor-memory</code> or reduce partition size.
Large shuffle spill	Stages tab — Shuffle Spill column	Not enough RAM: data spills to disk. Fix: more memory or fewer partitions.
Many empty tasks	Stages tab — Task count	Over-partitioned: too many partitions for dataset size. Fix: <code>coalesce()</code> .

Lab Task

In today's lab, you'll observe each of these in the real Spark Web UI while running experiments

The Problem: Repeated HDFS Reads



3 actions = 3 full HDFS reads! Each re-executes the entire lineage.

Why?

Spark is **lazy** — it doesn't store intermediate results unless you ask. Every action recomputes from scratch by replaying the DAG lineage.

Why Does Spark Re-Read Data?



Spark's Lazy Evaluation Model

- Spark builds a **DAG** (Directed Acyclic Graph) of transformations — but does **nothing** until an action is called
- Each action triggers a **complete re-execution** of the DAG from the source (HDFS)
- This is by design: Spark doesn't know in advance which intermediate results you'll reuse



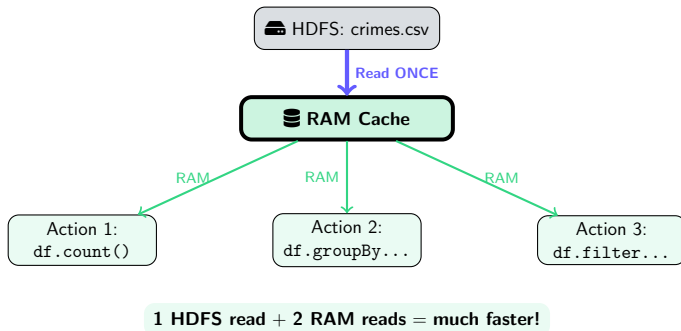
The Cost in Practice

- `df.count()` — reads all 10,000 rows from HDFS, counts them
- `df.groupBy(...).show()` — reads the **same** 10,000 rows again from HDFS
- `df.filter(...).count()` — reads them a **third time**
- With real datasets (millions of rows), each read can take **minutes**



The Insight

The Solution: `.cache()` and `.persist()`



Lab 07 Benchmark (Real Numbers)

Without cache: $21s + 21s + 21s = 63s$. With cache: $21s + 0.8s + 3.3s = 25s$. **Speedup:**
 $2.5\times$

How Caching Works Under the Hood

What Happens When You Call `.cache()`

- `df.cache()` **marks** the DataFrame for caching — nothing happens yet (lazy!)
- On the **first action** (e.g., `df.count()`), Spark reads from HDFS and stores the result in **executor RAM**
- On the **second action** (e.g., `df.groupBy(...)`), Spark reads from RAM — **no HDFS access**
- The first action takes the same time; subsequent actions are **10–100× faster**

Why the Diagram Shows “Read ONCE”

- The single arrow from HDFS to the RAM Cache represents the **one-time cost**
- The three green arrows from RAM to Actions represent **fast memory reads**
- Without caching: 3 HDFS reads (slow). With caching: 1 HDFS read + 2 RAM reads (fast)

Caching in Code

```
# .cache() = persist in RAM only (default)
df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)

df.cache()           # marks for caching (lazy!)
df.count()           # 1st action: reads HDFS, stores in RAM
df.groupBy("Primary Type").count().show() # from RAM!

# Unpersist when done (free memory)
df.unpersist()
```

Common Mistake

`df.cache()` does **NOT** cache immediately! Caching happens on the **first action** after the call.

Storage Levels — .persist()

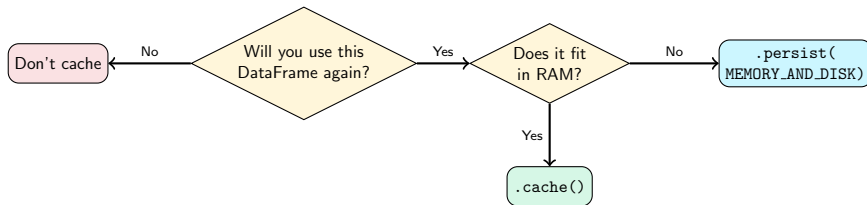
```
from pyspark import StorageLevel

df.persist(StorageLevel.MEMORY_AND_DISK)  # spill to disk
df.persist(StorageLevel.DISK_ONLY)        # disk only
```

Level	RAM	Disk	Replicated	Use Case
MEMORY_ONLY	✓	✗	No	Default (.cache()). Fastest.
MEMORY_AND_DISK	✓	✓	No	Large data that may not fit
DISK_ONLY	✗	✓	No	Very large data, save RAM
MEMORY_ONLY_2	✓	✗	2×	Critical data, fault tolerance

Rule of Thumb

When to Cache — Decision Flowchart



Cache These

- DataFrames used by **multiple actions**
- **Training data** before `model.fit()` (iterative)
- DataFrames after **expensive joins/aggregations**

Walking Through the Caching Decision

? Question 1: "Will I use this DataFrame again?"

- If you only run **one action** on a DataFrame → **don't cache** (waste of memory)
- If you run **multiple actions** (count, groupBy, filter, join) → proceed to Question 2
- ML training is **always** multiple passes → always cache training data

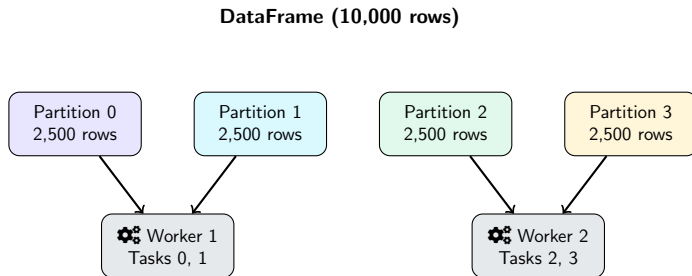
? Question 2: "Does it fit in RAM?"

- Check the Storage tab in the Web UI to see cached size
- If DataFrame fits in RAM → use `.cache()` (equivalent to `MEMORY_ONLY`)
- If DataFrame is too large → use `.persist(MEMORY_AND_DISK)` to spill excess to disk
- **Never** cache a DataFrame you read once and throw away

Don't Forget to Unpersist!

Call `df.unpersist()` when you no longer need the cached data. Cached DataFrames stay in

What Are Partitions?



1 partition = 1 task

Each task runs on one core of one executor.

```
# Check current partitions
df.rdd.getNumPartitions()
# Default from HDFS = block count
```

Understanding Partitions — Why They Matter



Partitions = Parallelism

- A partition is a **chunk** of your DataFrame that can be processed independently
- Each partition becomes **one task** — and tasks run in parallel on executor cores
- **4 partitions on 4 cores** = all cores busy simultaneously = maximum parallelism



The Goldilocks Problem

- **Too few partitions** (e.g., 1): only 1 core works, other 3 sit idle → slow
- **Too many partitions** (e.g., 200): scheduling overhead per task, mostly empty tasks → slow
- **Just right**: 2–4× total cores. Our cluster: 4 cores → **8–16 partitions**

i How Partitions Are Decided

Default partition count comes from the data source: **HDFS** → one partition per block (128 MB blocks); **CSV** → depends on file splits. You can override with `repartition(n)` or

repartition() vs coalesce()

repartition(n)	coalesce(n)
Full shuffle (expensive)	No shuffle (narrow dependency)
Can increase or decrease	Can only decrease
Even distribution guaranteed	Some partitions may be larger
Use for: heavy computation	Use for: writing output files

```
# Increase partitions (full shuffle)
df = df.repartition(8)

# Decrease partitions (no shuffle -- just merge)
df_out = df.coalesce(2)

# Partition by column (co-locate related rows)
df = df.repartition(4, "District")
```

Sweet Spot

Set partitions to **2–4 × total executor cores**. Our cluster: 4 cores → 8 partitions is a good default.

Shuffle Partitions Configuration

After any **shuffle operation** (GROUP BY, JOIN, etc.), Spark creates `spark.sql.shuffle.partitions` partitions — default is **200**.

```
# Default: 200 partitions after any shuffle  
# BAD for small clusters: 200 tasks on 4 cores!  
spark.conf.set("spark.sql.shuffle.partitions", 8)
```

200 partitions on 4 cores



Most tasks empty or tiny → overhead!

8 partitions on 4 cores



Each core stays busy → efficient!

Rule

Set `spark.sql.shuffle.partitions` to **2–4 × total cores**. Always include this in your `spark-submit` command.

Why 200 Default Partitions Is a Problem

What Are Shuffle Partitions?

- After any **wide transformation** (`groupBy`, `join`, `orderBy`), Spark redistributes data
- The number of output partitions is controlled by `spark.sql.shuffle.partitions`
- Default value: **200** — designed for large production clusters, not small lab clusters

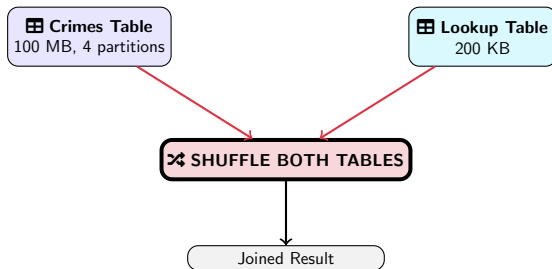
The Math for Our Cluster

- $200 \text{ partitions} \div 4 \text{ cores} = \mathbf{50 \text{ rounds}}$ of task execution
- Most partitions will be **nearly empty** ($10,000 \text{ rows} \div 200 = 50 \text{ rows each}$)
- The overhead of scheduling 200 tasks far exceeds the actual computation
- Setting to **8 partitions**: $8 \div 4 \text{ cores} = 2 \text{ rounds}$, each with meaningful data

Always Set This!

Add `--conf spark.sql.shuffle.partitions=8` to every `spark-submit` command on our

The Shuffle Join Problem



Shuffles 100 MB + 200 KB across network = WASTEFUL

Problem

Default join shuffles **both** tables. Shuffling 100 MB of crimes data just to join with a 200 KB lookup table is extremely wasteful.

Why Shuffling Both Tables Is Wasteful

What Happens During a Shuffle Join

- Spark must ensure matching keys end up on the **same executor**
- To do this, it **redistributes (shuffles) both tables** across the network by join key
- The 100 MB Crimes table is split and sent across workers — **expensive network I/O**
- The 200 KB Lookup table is also shuffled — **unnecessary for such a tiny table**

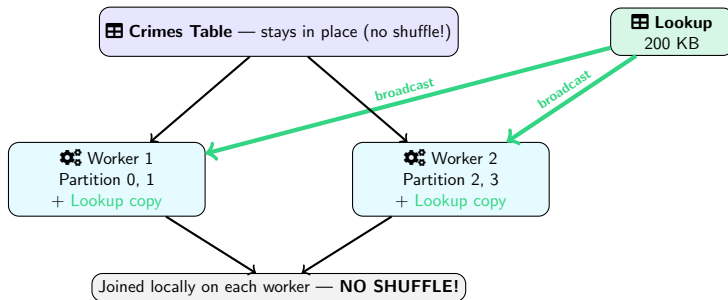
The Waste Visualized

- Network transfer: $100 \text{ MB} + 200 \text{ KB} \approx \mathbf{100 \text{ MB}}$ moved across the network
- The lookup table is 0.2% of the total data but causes 100% of the join to shuffle
- On a real cluster with GBs of data, shuffle joins can take **minutes to hours**

The Key Question

If the lookup table is only 200 KB, why not just **copy it to every worker** and let each worker

Broadcast Join — The Solution



Result

Small table (<10 MB) is copied to every executor. Join happens **locally** — zero shuffle of the large table!

How Broadcast Join Eliminates Shuffle

'A' The Broadcast Strategy

- The small Lookup table (200 KB) is **copied entirely** to every executor's memory
- Each executor now has the full lookup table **locally available**
- The large Crimes table **stays in place** — its partitions don't move at all
- Each executor joins its local partitions with the local copy of the lookup table

📊 Performance Comparison

- **Shuffle join**: moves 100 MB across the network → slow
- **Broadcast join**: copies only 200 KB to 2 executors (400 KB total) → fast
- Network savings: **99.6% less data** transferred
- No shuffle stage needed → fewer stages → faster execution plan

⚠️ When NOT to Broadcast

Broadcast Join in Code

```
from pyspark.sql.functions import broadcast

# Small lookup table
district_df = spark.createDataFrame([
    (1, "Central"), (2, "Wentworth"),
    (8, "Chicago Lawn"), (11, "Harrison")
], ["DistrictID", "DistrictName"])

# WITHOUT broadcast (shuffles both tables)
result = df.join(district_df,
    df["District"] == district_df["DistrictID"])

# WITH broadcast (sends lookup to all executors)
result = df.join(broadcast(district_df),
    df["District"] == district_df["DistrictID"])
```

Auto-Broadcast

Spark auto-broadcasts tables < 10 MB
(configurable via `spark.sql.
autoBroadcastJoinThreshold`)

Verify with explain()

Look for BroadcastHashJoin
in `result.explain()`

Join Strategy Summary

Scenario	Strategy	Cost
Both tables large	Shuffle (Sort-Merge) Join	High (full shuffle)
One table small (<10 MB)	Broadcast Join	Low (no shuffle)
Same partition key	Co-partitioned join	Medium (pre-repartition)
Frequent joins on same key	Bucket join	Low (pre-bucketed)

💡 Key Insight

In Big Data courses, most “lookup” joins (district names, payment types, status codes) involve a **small dimension table**. Always use `broadcast()` for these!

Key Takeaways

- 1 **spark-submit**: deploy `.py` scripts to YARN (`client` for dev, `cluster` for production)
- 2 **Web UI**: Jobs $\rightarrow$ Stages $\rightarrow$ Tasks. Watch shuffle bytes, straggler tasks, GC time
- 3 **Caching**: `.cache()` keeps data in RAM; avoids repeated HDFS reads. First action is same speed; subsequent actions are 10–100 $\times$ faster
- 4 **Partitions**: set to 2–4 $\times$ total cores. Use `repartition()` to increase, `coalesce()` to decrease
- 5 **Broadcast joins**: send small tables to all executors; eliminates shuffle of the large table



The Three Levers of Spark Performance

Caching — Partitioning — Broadcast

What's Next

Week 8B: Spark MLlib — Machine Learning at Scale

- ML Pipeline API: Transformer, Estimator, Pipeline
- Feature engineering: StringIndexer, VectorAssembler
- Train a Random Forest to predict arrests in Chicago crimes
- Evaluate: AUC-ROC, accuracy, F1 score, confusion matrix

Today's Lab: Spark Performance Tuning

- Experiment 1: With vs without caching (3 actions)
- Experiment 2: Partition count (1, 4, 8, 16)
- Experiment 3: Broadcast vs shuffle join
- Record timings + Web UI screenshots

Milestone M3 Reminder

Due end of Week 8: RDD + DataFrame + MLlib analytics on Chicago Crimes.